Contents

AUGECOIN Technical Book

Version: unknown **Commit:** unknown **Generated:** 2026-08-19 13:26 UTC **Graphify:** graphify-data-present **Rust:** rustc 1.97.1 (8bab26f4f 2026-07-14)

> This document is automatically generated from the AUGECOIN codebase, Graphify knowledge graph, architecture docs and specifications. Intended for external audits, enterprise review, researchers and AI systems.

Chapter 1 — Overview

Executive Summary

AUGECOIN is a Proof-of-Authority blockchain built in Rust, featuring: - **Hybrid signatures**: Ed25519 + CRYSTALS-Dilithium (post-quantum) - **BLAKE3-512** hashing (XOF mode) - **SafeBox** incremental state model with deterministic snapshots - **PoA consensus** with 2/3+1 quorum and round-robin leader selection - **Native numbered accounts** (AUGEID) with marketplace, gifts and name resolution - **Total supply**: 750,000,000 AUGE (75 trillion augesat), emitted over 50 years

Graphify Analysis

Metric	Value
Nodes	3713
Edges	8310
Communities	217
Extraction Confidence	99%

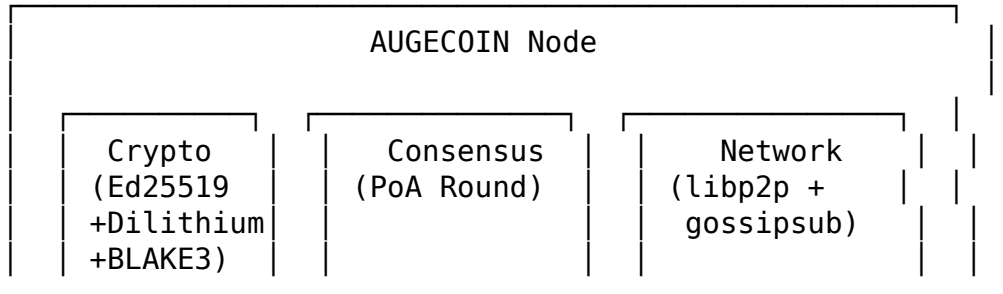
God Nodes (Core Abstractions)

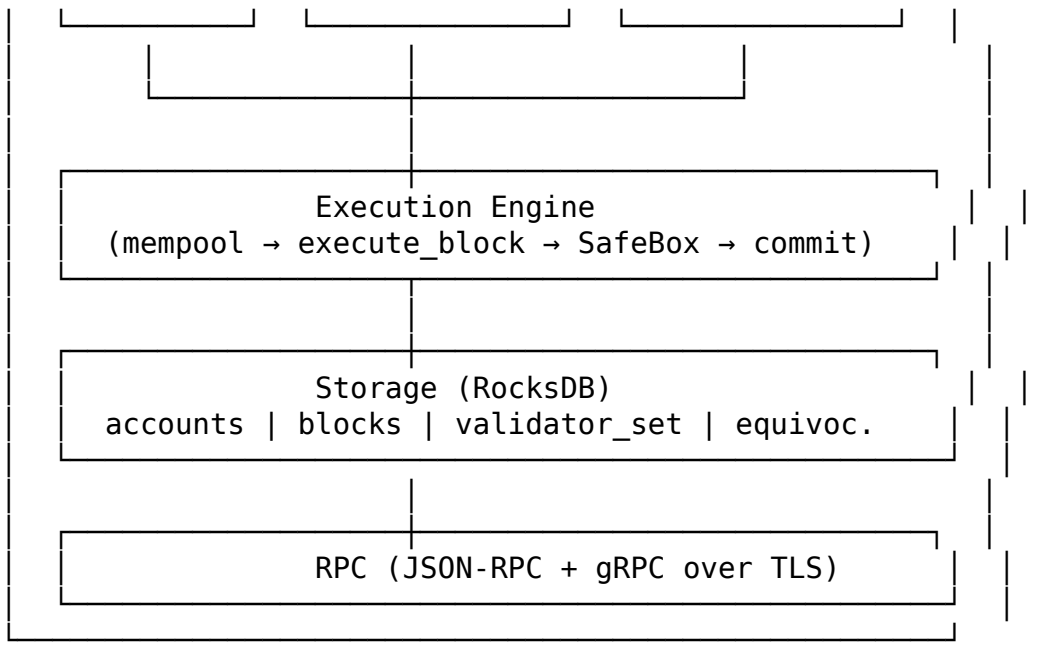
Rank	Name	Edges
1	`Storage`	72
2	`AppState`	56
3	`AUGECOIN — PROMPTS.md`	48
4	`Mempool`	43
5	`Account`	40
6	`esc()`	39
7	`StorageError`	38
8	`ConsensusEngine`	35
9	`esc()`	29
10	`fmtNum()`	27

Top Communities

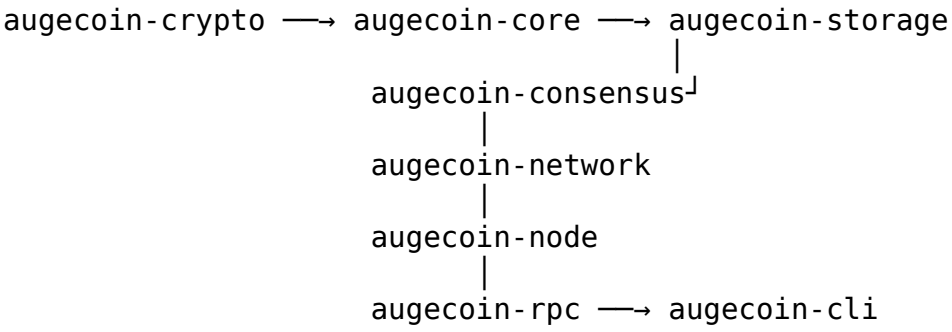
- **explorer/assets/js/utils.js** (? nodes, cohesion: 0.050)
- **Storage** (? nodes, cohesion: 0.060)
- **execution.rs** (? nodes, cohesion: 0.050)
- **src/index.ts** (? nodes, cohesion: 0.060)
- **operacional/assets/js/app.js** (? nodes, cohesion: 0.050)
- **augecoin-cli/src/commands.rs** (? nodes, cohesion: 0.060)
- **Mempool** (? nodes, cohesion: 0.100)
- **augecoin-network/src/lib.rs** (? nodes, cohesion: 0.080)
- **augecoin-rpc/src/lib.rs** (? nodes, cohesion: 0.090)
- **operation.rs** (? nodes, cohesion: 0.110)

Architecture Summary





Crate Dependency Graph



Chapter 2 — Architecture

Rust Workspace

The project is organized as a Cargo workspace with the following crates:

Crate	Entry	Sources	Tests
augecoin-bench	lib.rs	10 src	1 tests
augecoin-cli	main.rs	4 src	0 tests
augecoin-consensus	lib.rs	5 src	2 tests
augecoin-core	lib.rs	12 src	1 tests
augecoin-crypto	lib.rs	6 src	0 tests
augecoin-network	lib.rs	3 src	0 tests
augecoin-node	lib.rs	8 src	8 tests

Crate	Entry	Sources	Tests
`augecoin-rpc`	lib.rs	4 src	0 tests
`augecoin-storage`	lib.rs	3 src	0 tests

Crate Responsibilities

augecoin-crypto

Cryptographic primitives: Ed25519+Dilithium hybrid signatures, BLAKE3-512 hashing, HD key derivation (HKDF-SHA3-512), Bech32m address encoding, WASM bindings.

augecoin-core

Core data types: AugeAccount, Transaction, Block, BlockHeader, Operation, SafeBox, emission logic, mempool. Protocol-level serialization.

augecoin-storage

RocksDB persistence: column families for accounts, blocks, tx_index, validator_set, equivocation_proofs. State root computation (Merkle), checkpoints, snapshots.

augecoin-consensus

PoA consensus engine: ValidatorSet, round state machine (Propose→Collect→Commit), quorum verification (2/3+1), equivocation detection, leader selection.

augecoin-network

libp2p networking: Noise encryption, gossipsub for tx/block propagation, peer discovery (static bootnodes), sync manager, rate limiting.

augecoin-node

Node orchestration: block execution, mempool validation, genesis initialization, metrics (Prometheus), alert system, main entry point.

augecoin-rpc

JSON-RPC 2.0 + gRPC server over TLS: all blockchain endpoints, auth (API key), rate limiting, account/block/transaction queries.

augecoin-cli

CLI tool: validator management, status queries, earnings, security commands. Tab completion via clap, JSON and table output.

augecoin-bench

Benchmarks: block execution, consensus, storage, networking benchmarks.

Architecture Decision Records

The project maintains decisions in `docs/DECISIONS.md` with ADRs covering: - ADR-001: Cargo workspace structure - ADR-002: RocksDB as storage backend - ADR-003: Hybrid signature scheme (Ed25519 + Dilithium) - ADR-004: BLAKE3-512 for hashing - ADR-005: ClaimAccount auto-assignment to validator leader

---

Chapter 3 — SafeBox

> **Veja também:** Storage, Consenso, AUGEID

Executive Summary

The SafeBox is AUGECOIN's core state container — a deterministic, incrementally committed BTreeMap of accounts that forms the consensus state root.

Responsibility

Persist the full blockchain state (accounts, name index) in a deterministic, verifiable structure. Every block execution modifies the SafeBox atomically.

Architectural Flow

AUGECOIN — SafeBox Flow

> Fonte: `crates/augecoin-core/src/safe_box.rs`, `crates/augecoin-storage/src/lib.rs`, `crates/augecoin-node/src/execution.rs`. Nós do grafo: `SafeBox`, `SafeboxCache`, `compute_safe_box_hash`, `commit_safebox_incremental`.

Estrutura

```
SafeBox {
 header: SafeBoxHeader { protocol, start_block, end_block, blocks_count, safe_box_has
  accounts: BTreeMap<u64, Account>           // ordenado por número
 name_index: BTreeMap<String, u64>      // nome → número de conta (índice de nomes)
}
```

Ciclo de vida do AUGEID (AccountState)

```
Unknown → (emissão do bloco) → Reserved → (venda/doação/ativação) → Owned → Normal
                                                  ↘ GiftPending (aguardando aceite)
```

Estado	Descrição
`Reserved`	Emitido ao líder do bloco; sem chave definitiva; não envia/recebe AUGE
`Owned`	Tem proprietário (`account_key`, `n_operation=0`, `account_seal`)
`ForSale`	Anunciado no marketplace (`sale_index`)
`GiftPending`	Doação pendente (`gift_index`); aguarda aceite
`Normal`	Conta operacional

A numeração é determinística: **AUGEID = bloco × 10 + offset (0..9)**. Cada bloco emite exatamente 10 AUGEIDs, todos `Reserved` e pertencentes ao líder.

Hash de consenso

`compute_safe_box_hash()` = raiz Merkle (blake3_512) sobre: 1. `accounts.values().map(|a| a.hash())` — folha por conta; 2. `name_index` — `blake3(name || number)` por entrada.

- **Empty SafeBox → `[0u8; 64]`.**
- O hash é gravado no header do bloco como `initial_safe_box_hash` e realimentado no bloco seguinte (`prev_safe_box_hash`). **É parte do consenso.**
- `sale_index` e `gift_index` são índices residentes derivados (NÃO entram no hash).

Persistência incremental

Todas as escritas de um bloco (contas modificadas + bloco + altura + validator_set + snapshot periódico) são consolidadas num único `commit_block_atomic` (WriteBatch).

```
execute_block (por bloco)
  └─ commit_block_atomic(&modified, block, validator_set_bytes)
     ├─ WriteBatch: put_account(contas) + put_block + put_height + put_validator_set
     ├─ atualiza SafeboxCache residente (accounts + name_index + pubkey_index
     │   + sale_index + gift_index) — O(k)
     └─ snapshot completo periódico (intervalo configurável)
```

Índices residentes (O(1), sincronizados incrementalmente)

Índice	Mapeamento	Uso
`name_index`	nome → AUGEID	resolução `resolve_name("CarlosPay")`

Índice	Mapeamento	Uso
`pubkey_index`	pubkey → refcount	detecção de chave duplicada
`sale_index`	AUGEID → `SaleListing`	marketplace (`list_for_sale`)
`gift_index`	AUGEID → `GiftListing`	doações pendentes (`list_pending_gifts`)

Snapshot periódico

- Intervalo: `SAFEBOX_SNAPSHOT_INTERVAL_DEV = 100` / `_MAINNET = 1000` (override `AUGECOIN_SAFEBOX_SNAPSHOT_INTERVAL`).
- `flush_safebox_snapshot()` — força snapshot (shutdown/export/recovery).

Recuperação

- O hash é **sempre** reconstruído do CF `accounts` (fonte de verdade), nunca de um snapshot potencialmente atrasado — crash entre snapshots é seguro.

Invariantes

- `SafeBox.accounts` ≡ conteúdo do CF `accounts` após o commit.
- `name_index` ≡ derivado dos `account.name`.
- A emissão é fixa: exatamente 10 AUGEIDs `Reserved` por bloco, todos do líder.
- `CreateAccount` nunca cria número novo — apenas ativa um `Reserved` existente.

Main File

Arquivo: `crates/augecoin-core/src/safe_box.rs` **Comunidade Graphify:** 2 — “Storage” **God Node:** Storage (68 edges)

Source Code (excerpt)

```rust
use crate::account::Account;
use augecoin_crypto::hash::blake3_512;
use std::collections::BTreeMap;
use thiserror::Error;

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct MerkleProof {
    /// The account as it exists in the SafeBox.
    pub account: Account,
    /// Sibling hashes from leaf to root (each entry is the sibling at that level)
    pub siblings: Vec<[u8; 64]>,
    /// Index of the account leaf in the full leaf list (0-based).
    pub leaf_index: usize,
```

```rust
    /// Total number of leaves in the tree when the proof was generated.
    pub total_leaves: usize,
}

#[derive(Debug, Error)]
pub enum MerkleProofError {
    #[error("proof verification failed")]
    InvalidProof,
    #[error("account not found in safebox")]
    AccountNotFound,
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct SafeBoxHeader {
    pub protocol: u16,
    pub start_block: u64,
    pub end_block: u64,
    pub blocks_count: u64,
    pub safe_box_hash: [u8; 64],
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct SafeBox {
    pub header: SafeBoxHeader,
    pub accounts: BTreeMap<u64, Account>,
    pub name_index: BTreeMap<String, u64>,
}

#[derive(Debug, Error)]
pub enum SafeBoxError {
    #[error("account not found: {0}")]
    AccountNotFound(u64),
    #[error("invalid safe box hash")]
    InvalidHash,
}

impl SafeBoxHeader {
    pub fn new(protocol: u16, start_block: u64, end_block: u64) -> Self {
        SafeBoxHeader {
            protocol,
            start_block,
            end_block,
            blocks_count: end_block.saturating_sub(start_block),
            safe_box_hash: [0u8; 64],
        }
    }

    pub fn to_bytes(&self) -> Vec<u8> {
```

```rust
        let mut buf = Vec::new();
        buf.extend_from_slice(&self.protocol.to_be_bytes());
        buf.extend_from_slice(&self.start_block.to_be_bytes());
        buf.extend_from_slice(&self.end_block.to_be_bytes());
        buf.extend_from_slice(&self.blocks_count.to_be_bytes());
        buf.extend_from_slice(&self.safe_box_hash);
        buf
    }

    pub fn from_bytes(data: &[u8]) -> Option<Self> {
        if data.len() < 2 + 8 + 8 + 8 + 64 {
            return None;
        }
        let mut pos = 0;
        let protocol = u16::from_be_bytes([data[pos], data[pos + 1]]);
        pos += 2;
        let start_block = u64::from_be_bytes(data[pos..pos + 8].try_into().unwrap(
        pos += 8;
        let end_block = u64::from_be_bytes(data[pos..pos + 8].try_into().unwrap())
        pos += 8;
        let blocks_count = u64::from_be_bytes(data[pos..pos + 8].try_into().unwrap
        pos += 8;
        let safe_box_hash: [u8; 64] = data[pos..pos + 64].try_into().unwrap();

        Some(SafeBoxHeader {
            protocol,
            start_block,
            end_block,
            blocks_count,
            safe_box_hash,
        })
    }
}

impl SafeBox {
    pub fn new(protocol: u16, start_block: u64) -> Self {
        SafeBox {
            header: SafeBoxHeader::new(protocol, start_block, start_block),
            accounts: BTreeMap::new(),
            name_index: BTreeMap::new(),
        }
    }

    pub fn get_account(&self, account_number: u64) -> Option<&Account> {
        self.accounts.get(&account_number)
    }

    pub fn get_account_mut(&mut self, account_number: u64) -> Option<&mut Account>
```

```
        self.accounts.get_mut(&account_number)
    }

    pub fn add_account(&mut self, account: Account) {
        // Remove old name entry for this account if it had one
        if let Some(old) = self.accounts.get(&account.account_number) {
            if let Some(ref old_name) = old.name {
                self.name_index.remove(old_name);
            }
        }
        // Insert new name if account has one
        if let Some(ref name) = account.name {
            self.name_index.insert(name.clone(), account.account_number);
        }
        self.accounts.insert(account.account_number, account);
    }

    pub fn account_count(&self) -> u64 {
        self.accounts.len() as u64
    }

    pub fn max_account_number(&self) -> u64 {
        self.accounts.keys().last().copied().unwrap_or(0)
    }

    pub fn is_name_taken(&self, name: &str) -> Option<u64> {
        self.name_index.get(name).copied()
    }

    pub fn compute_safe_box_hash(&self) -> [u8; 64] {
        if self.accounts.is_empty() {
            return [0u8; 64];
        }
        let mut hashes: Vec<[u8; 64]> = self.accounts.values().map(|a| a.hash()).c
        // Include name_index entries in the hash for consensus
        for (name, number) in &self.name_index {
            let mut data = Vec::new();
            data.extend_from_slice(name.as_bytes());
            data.extend_from_slice(&number.to_be_bytes());
            hashes.push(blake3_512(&data));
        }
// ... (492 lines total)
```

Riscos

- Memory usage scales with account count (BTreeMap in memory)
- Recovery depends on RocksDB integrity

- Snapshot interval affects recovery time

Referências Cruzadas

- **Storage** (Chapter 5): RocksDB persistence of SafeBox state
- **Consenso** (Chapter 4): SafeBox hash included in block header
- **AUGEID** (Chapter 9): Account lifecycle within SafeBox

Chapter 4 — Consensus

> **Veja também:** SafeBox, Rede, RPC

Executive Summary

AUGECOIN uses Proof of Authority (PoA) with round-robin leader selection, 2/3+1 quorum requirement, and automatic view-change on timeout.

Responsibility

Coordinate block proposal, validation, signing and finalization among authorized validators.

Architectural Flow

AUGECOIN — Consensus Flow (PoA)

> Fonte: `crates/augecoin-consensus/*`, `crates/augecoin-node/src/consensus.rs`, `crates/augecoin-node/src/main.rs`. Nós do grafo: `ConsensusEngine`, `RoundState`, `ValidatorSet`, `quorum_threshold`, `EquivocationProof`.

Modelo

- **PoA (Proof of Authority)** com líder rotativo round-robin.
- `ValidatorSet.active_validators()` ordena os validadores ativos.
- Líder da altura H: `leader_for(H, round, set)` = `active[(H+1+round) % len]`.
- Quórum: `quorum_threshold(n)` = 2/3 (ver `quorum.rs`).

Fases de um round (`round.rs`)

1. **Propose** — líder constrói bloco (`ConsensusEngine::build_block`) com `initial_safe_box_hash` (hash do SafeBox anterior) e assina.
2. **Collect** — `add_signature` acumula assinaturas dos demais validadores.
3. **Commit** — `try_commit` quando assinaturas ≥ quórum.
4. **Timeout** — `check_timeout` dispara `view_change` para o próximo líder.

Eventos de rede (`ConsensusEvent`, `ConsensusTransport`)

- `BlockProposal`, `BlockResponse`, `BlockRequest`, `StatusRequest`, `RoundChange`, `CommitNotification`, operações gossiped.
- Transporte: libp2p gossipsub (mesh de consenso) + Kademlia (descoberta).

Verificação de bloco (`execution.rs::verify_block_quorum`)

1. Rejeita timestamp futuro além de `CT_MAX_FUTURE_BLOCK_TIMESTAMP_SECONDS`.
2. Verifica assinatura do líder (Ed25519) sobre `block.hash()`.
3. Verifica assinaturas de quórum dos validadores ativos.
4. Valida `operations_hash` == Merkle das operações do bloco.

Fluxo de finalização (visão `main.rs`)

```
Idle → (se é líder) build_block → AwaitingSignatures
     → (threshold atingido) execute_block → Executed
     → grava bloco + altura + SafeBox incremental + prune
     → sync_validator_set_from_storage → próximo round
```

- Catch-up: `apply_catchup_block` (blocos recebidos fora de ordem são aplicados após verificação de quórum).
- Equivocação: `detect_equivocation` gera `EquivocationProof` persistido em `equivocation_proofs`.

Determinismo

- `execute_block` é determinístico: mesmo bloco + mesmo estado → mesmo `safe_box_hash` (validado por `determinism_test.rs`).
- `compute_safe_box_hash` (Merkle) não mudou; a refatoração do SafeBox afetou apenas a persistência local, nunca o hash de consenso.

Key Files

quorum.rs

Comunidade: 82 — “quorum.rs” **God Node:** quorum.rs

```rust
use crate::validator::{ValidatorInfo, ValidatorSet};
use augecoin_crypto::signature::HybridSignature;

pub fn quorum_threshold(validator_count: u64) -> u64 {
    if validator_count == 0 {
        return 0;
    }
    (validator_count * 2) / 3 + 1
}

pub fn verify_quorum(
```

```rust
    signatures: &[HybridSignature],
    validator_set: &ValidatorSet,
    block_hash: &[u8; 64],
) -> bool {
    let active = validator_set.active_validators();
    let threshold = quorum_threshold(active.len() as u64);

    let mut valid_count: u64 = 0;

    for sig in signatures {
        for validator in &active {
            if sig.verify_validator(validator, block_hash) {
                valid_count += 1;
                break;
            }
        }
    }

    valid_count >= threshold
}

trait SignatureExt {
    fn verify_validator(&self, validator: &ValidatorInfo, message: &[u8]) -> bool;
}

impl SignatureExt for HybridSignature {
    fn verify_validator(&self, validator: &ValidatorInfo, message: &[u8]) -> bool {
        let pk = ed25519_dalek::VerifyingKey::from_bytes(&validator.ed25519_public_
        self.verify(&pk, message)
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::validator::ValidatorInfo;
    use augecoin_crypto::hdkeys::HdWallet;
    use augecoin_crypto::signature::HybridKeyPair;

    #[test]
    fn quorum_3_of_4() {
        assert_eq!(quorum_threshold(4), 3);
    }

    #[test]
    fn quorum_2_of_2() {
        assert_eq!(quorum_threshold(2), 2);
    }
```

```rust
#[test]
fn quorum_5_of_7() {
    assert_eq!(quorum_threshold(7), 5);
}

#[test]
fn quorum_1_of_1() {
    assert_eq!(quorum_threshold(1), 1);
}

#[test]
fn exactly_quorum_finalizes() {
    let admin = HybridKeyPair::generate();
    let validators: Vec<ValidatorInfo> = (0..4).map(make_test_validator).colle
    let set = ValidatorSet::new(admin.verifying_key(), validators);

    let block_hash = [1u8; 64];
    let sigs: Vec<HybridSignature> = (0..3)
        .map(|i| {
            let kp = make_test_keypair(i);
            kp.sign(&block_hash)
        })
        .collect();

    assert!(verify_quorum(&sigs, &set, &block_hash));
}

#[test]
fn less_than_quorum_does_not_finalize() {
    let admin = HybridKeyPair::generate();
    let validators: Vec<ValidatorInfo> = (0..4).map(make_test_validator).colle
    let set = ValidatorSet::new(admin.verifying_key(), validators);

    let block_hash = [2u8; 64];
    let sigs: Vec<HybridSignature> = (0..2)
        .map(|i| {
            let kp = make_test_keypair(i);
            kp.sign(&block_hash)
        })
        .collect();

    assert!(!verify_quorum(&sigs, &set, &block_hash));
}

fn make_test_keypair(index: u64) -> HybridKeyPair {
    let seed = [index as u8; 64];
    HdWallet::from_seed(&seed).derive_keypair(0)
```

```
    }

    fn make_test_validator(index: u64) -> ValidatorInfo {
        let kp = make_test_keypair(index);
        let vk = kp.verifying_key();
        let mut ed = [0u8; 32];
        ed.copy_from_slice(&vk.to_bytes());
        ValidatorInfo::new_active(index, ed)
    }
}
```

round.rs

Comunidade: 42 — “round.rs”

```
use crate::validator::ValidatorSet;
use augecoin_core::block::OperationBlock;
use augecoin_crypto::signature::HybridSignature;
use thiserror::Error;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum RoundPhase {
    Propose,
    Verify,
    Sign,
    Commit,
}

#[derive(Debug, Clone)]
pub struct RoundState {
    pub height: u64,
    pub phase: RoundPhase,
    pub leader_id: u64,
    pub proposed_block: Option<OperationBlock>,
    pub collected_signatures: Vec<HybridSignature>,
    pub timeout_seconds: u64,
    pub phase_entered_at: u64,
}

#[derive(Debug, Error)]
pub enum RoundError {
    #[error("invalid phase transition from {from:?} to {to:?}")]
    InvalidTransition { from: RoundPhase, to: RoundPhase },
    #[error("no block proposed")]
    NoBlockProposed,
}

impl RoundState {
```

```rust
    pub fn new(height: u64, leader_id: u64, timeout_seconds: u64, now: u64) -> Sel
        RoundState {
            height,
            phase: RoundPhase::Propose,
            leader_id,
            proposed_block: None,
            collected_signatures: Vec::new(),
            timeout_seconds,
            phase_entered_at: now,
        }
    }

    pub fn propose_block(&mut self, block: OperationBlock) -> Result<(), RoundErro
        if self.phase != RoundPhase::Propose {
            return Err(RoundError::InvalidTransition {
                from: self.phase,
                to: RoundPhase::Verify,
            });
        }
        self.proposed_block = Some(block);
        self.phase = RoundPhase::Verify;
        Ok(())
    }

    pub fn verify_and_advance(&mut self, is_valid: bool) -> Result<(), RoundError>
        if self.phase != RoundPhase::Verify {
            return Err(RoundError::InvalidTransition {
                from: self.phase,
                to: RoundPhase::Sign,
            });
        }
        if !is_valid {
            return Ok(());
        }
        self.phase = RoundPhase::Sign;
        Ok(())
    }

    pub fn add_signature(&mut self, signature: HybridSignature) {
        self.collected_signatures.push(signature);
    }

    pub fn try_commit(&mut self, validator_set: &ValidatorSet) -> Result<bool, Rou
        if self.phase != RoundPhase::Sign {
            return Err(RoundError::InvalidTransition {
                from: self.phase,
                to: RoundPhase::Commit,
            });
```

```
        }

        let active_count = validator_set.active_validators().len() as u64;
        if active_count == 0 {
            return Ok(false);
        }

        let quorum_threshold = crate::quorum::quorum_threshold(active_count);
        if self.collected_signatures.len() as u64 >= quorum_threshold {
            self.phase = RoundPhase::Commit;
            return Ok(true);
        }

        Ok(false)
    }

    pub fn check_timeout(&self, now: u64) -> bool {
        now >= self.phase_entered_at + self.timeout_seconds
    }

    pub fn view_change(&mut self, next_leader_id: u64, now: u64) {
        self.leader_id = next_leader_id;
        self.phase = RoundPhase::Propose;
        self.proposed_block = None;
        self.collected_signatures.clear();
        self.phase_entered_at = now;
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use augecoin_core::block::OperationBlockHeader;
    use augecoin_core::operation::{
        Operation, OperationPayload, OperationType, ReceiverInfo, SenderInfo,
    };

    fn dummy_sig() -> HybridSignature {
        HybridSignature { bytes: [7u8; 64] }
    }

    fn dummy_block(leader_id: u64) -> OperationBlock {
        OperationBlock {
            header: OperationBlockHeader {
                block_number: 1,
                account_key: [0u8; 32],
                reward: 100,
                fee: 0,
```

```
                protocol_version: 5,
                protocol_available: 6,
                timestamp: 1000,
                initial_safe_box_hash: [0u8; 64],
                operations_hash: [0u8; 64],
                block_payload: vec![],
                proof_of_work: [0u8; 32],
                previous_proof_of_work: [0u8; 32],
                leader_id,
                chain_id: 1,
            },
            operations: vec![Operation {
                op_type: OperationType::Transaction,
                chain_id: 1,
                payload: OperationPayload::Transaction {
                    senders: vec![SenderInfo {
                        account: 1,
                        n_operation: 0,
                        amount: 100,
                        payload: vec![],
                    }],
// ... (212 lines total)
```

validator.rs

Comunidade: 20 — “validator.rs”

```rust
use augecoin_core::operation::{Operation, OperationPayload, ValidatorAdminOp};
use augecoin_crypto::signature::HybridPublicKey;
use thiserror::Error;

const ED25519_PK_LEN: usize = 32;

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ValidatorStatus {
    PendingActivation { activation_height: u64 },
    Active,
    Inactive,
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ValidatorInfo {
    pub id: u64,
    pub ed25519_public_key: [u8; ED25519_PK_LEN],
    pub status: ValidatorStatus,
}

#[derive(Debug, Clone)]
```

```rust
pub struct ValidatorSet {
    admin_public_key: HybridPublicKey,
    validators: Vec<ValidatorInfo>,
    next_validator_id: u64,
}

#[derive(Debug, Error)]
pub enum ValidatorSetError {
    #[error("invalid admin signature")]
    InvalidAdminSignature,
    #[error("validator already exists")]
    ValidatorAlreadyExists,
    #[error("validator not found")]
    ValidatorNotFound,
    #[error("no active validators")]
    NoActiveValidators,
    #[error("cannot remove last active validator")]
    CannotRemoveLastActive,
    #[error("{0}")]
    InvalidActivation(String),
}

impl ValidatorSet {
    pub fn new(admin_public_key: HybridPublicKey, initial_validators: Vec<Validato
        let mut next_id = 0u64;
        for v in &initial_validators {
            if v.id >= next_id {
                next_id = v.id + 1;
            }
        }
        ValidatorSet {
            admin_public_key,
            validators: initial_validators,
            next_validator_id: next_id,
        }
    }

    pub fn admin_public_key(&self) -> &HybridPublicKey {
        &self.admin_public_key
    }

    pub fn validators(&self) -> &[ValidatorInfo] {
        &self.validators
    }

    pub fn active_validators(&self) -> Vec<&ValidatorInfo> {
        self.validators
            .iter()
```

```rust
        .filter(|v| v.status == ValidatorStatus::Active)
        .collect()
}

pub fn leader_for_height(&self, height: u64) -> Result<&ValidatorInfo, Validat
    let active = self.active_validators();
    if active.is_empty() {
        return Err(ValidatorSetError::NoActiveValidators);
    }
    let index = (height as usize) % active.len();
    Ok(active[index])
}

pub fn apply_validator_admin(
    &mut self,
    op: &Operation,
    current_height: u64,
) -> Result<(), ValidatorSetError> {
    let va_op = match &op.payload {
        OperationPayload::ValidatorAdmin(op) => op,
        _ => return Err(ValidatorSetError::InvalidAdminSignature),
    };

    let message = op.to_bytes_stripped();
    let admin_pk = &self.admin_public_key;

    let all_ok = op
        .signatures
        .iter()
        .any(|sig| sig.verify(admin_pk, &message));
    if !all_ok {
        return Err(ValidatorSetError::InvalidAdminSignature);
    }

    match va_op {
        ValidatorAdminOp::Add {
            ed25519_public_key,
            activation_height,
            ..
        } => {
            let id = self.next_validator_id;
            self.next_validator_id += 1;

            let status = if *activation_height <= current_height {
                ValidatorStatus::Active
            } else {
                ValidatorStatus::PendingActivation {
                    activation_height: *activation_height,
```

```
                    }
                };

                self.validators.push(ValidatorInfo {
                    id,
                    ed25519_public_key: *ed25519_public_key,
                    status,
                });
            }
            ValidatorAdminOp::Remove {
                validator_id,
                activation_height,
                ..
            } => {
                if *activation_height > current_height {
                    return Err(ValidatorSetError::InvalidActivation(
                        "Remove operation requires activation_height <= current_he
                    ));
                }
                if !self.validators.iter().any(|v| v.id == *validator_id) {
                    return Err(ValidatorSetError::ValidatorNotFound);
                }
                if self.active_validators().len() <= 1
                    && self
                        .validators
                        .iter()
                        .any(|v| v.id == *validator_id && v.status == ValidatorSta
                {
                    return Err(ValidatorSetError::CannotRemoveLastActive);
                }
                self.validators.retain(|v| v.id != *validator_id);
            }
            ValidatorAdminOp::Activate {
// ... (576 lines total)
```

Comparação: Protocolo Antigo vs Novo

Aspect	Original	Current
Leader selection	Random	Round-robin deterministic
Quorum	Fixed 3/4	2/3+1 (scalable)
Timeout	60s	120s (2x block time)
Equivocation	Detected, not proven	On-chain proof generation

Riscos

- Single admin key for validator management (centralization)

- No slashing mechanism beyond admin removal
- View-change depends on timeout accuracy

Referências Cruzadas

- **SafeBox** (Chapter 3): Initial safe box hash in block proposal
- **Rede** (Chapter 6): Consensus transport over libp2p
- **RPC** (Chapter 7): Validator set queries

Chapter 5 — Storage

> **Veja também:** SafeBox, Consenso

Executive Summary

AUGECOIN uses RocksDB as its persistent storage backend with 5 column families for different data types. All block writes are atomic via WriteBatch.

Responsibility

Provide durable, consistent storage for accounts, blocks, validator set, transaction index and equivocation proofs.

RocksDB Column Families

CF Name	Content	Access Pattern
`accounts`	AugeAccount serialized	Read/write per block
`blocks`	Block headers + bodies	Write once, read often
`tx_index`	Transaction hash → block	Write once, index lookup
`validator_set`	Active validator set	Write per governance change
`equivocation_proofs`	EquivocationProof	Write on detection

Atomic Writes (WriteBatch)

All state changes for a block are consolidated in a single `WriteBatch`:

```
commit_block_atomic:
  ├─ put_account(modified_accounts)
  ├─ put_block(block)
  ├─ put_height(height)
  ├─ put_validator_set(validator_set)
  └─ flush (WAL)
```

WAL Growth Solution

The Write-Ahead Log (WAL) growth issue was resolved by: 1. Configuring `set_max_total_wal_size` to limit WAL disk usage 2. Periodic compaction triggers after checkpoint snapshots 3. `set_keep_log_num` for WAL file retention control

State Root (Merkle Tree)

use augecoin_core::account::Account; use augecoin_core::hash::blake3_512 as hash; use rocksdb::{IteratorMode, DB}; use std::collections::BTreeMap;

const EMPTY_STATE_ROOT: [u8; 64] = [0u8; 64];

pub fn compute_state_root(db: &DB, cf_accounts: &rocksdb::ColumnFamilyRef) -> [u8; 64] { let mut accounts: BTreeMap<u64, Account> = BTreeMap::new();

```
let iter = db.iterator_cf(cf_accounts, IteratorMode::Start);
for item in iter {
    let (key, value) = item.expect("rocksdb iteration error
```

Checkpoints

use crate::{Storage, CF_ACCOUNTS}; use augecoin_core::account::Account; use rocksdb::IteratorMode; use std::collections::HashMap;

const CHECKPOINT_VERSION: u8 = 1;

#[derive(Debug)] pub struct AccountSnapshot { height: u64, accounts: HashMap<u64, Account>, }

impl AccountSnapshot { pub fn height(&self) -> u64 { self.height } pub fn len(&self) -> usize { self.accounts.len() } pub fn is_empty(&self) -> bool { self.accounts.is_empty() }

Main Storage Implementation

```rust
pub mod checkpoint;
pub mod state_root;

use augecoin_core::account::Account;
use augecoin_core::block::OperationBlock;
use augecoin_core::safe_box::SafeBox;
use rocksdb::{ColumnFamilyDescriptor, DBCompressionType, Options, WriteBatch, DB};
use std::collections::{BTreeMap, HashMap};
use std::path::{Path, PathBuf};
use std::sync::Mutex;
use thiserror::Error;

pub(crate) const CF_ACCOUNTS: &str = "accounts";
const CF_BLOCKS: &str = "blocks";
const CF_SAFEBOX: &str = "safebox";
```

```rust
const CF_OP_INDEX: &str = "op_index";
const CF_VALIDATOR_SET: &str = "validator_set";
const CF_EQUIVOCATION_PROOFS: &str = "equivocation_proofs";
const CF_FAUCET_CLAIMS: &str = "faucet_claims";

const ALL_COLUMN_FAMILIES: &[&str] = &[
    CF_ACCOUNTS,
    CF_BLOCKS,
    CF_SAFEBOX,
    CF_OP_INDEX,
    CF_VALIDATOR_SET,
    CF_EQUIVOCATION_PROOFS,
    CF_FAUCET_CLAIMS,
];

// RocksDB tuning — see DECISIONS: disk growth was dominated by full SafeBox
// rewrites every block plus WAL-driven flushes. These options reduce write
// amplification and bound the info log.
const WAL_SIZE: u64 = 1024 * 1024 * 1024; // 1 GiB max total WAL
const MAX_LOG_FILE_SIZE: usize = 64 * 1024 * 1024; // rotate info LOG at 64 MiB
const KEEP_LOG_FILE_NUM: usize = 10; // keep at most 10 info LOG files
const LOG_FILE_TIME_TO_ROLL: usize = 86400; // seconds (1 day)
const WRITE_BUFFER_SIZE: usize = 64 * 1024 * 1024; // 64 MiB memtable
const MAX_WRITE_BUFFER_NUMBER: i32 = 3;
const MIN_WRITE_BUFFER_NUMBER_TO_MERGE: i32 = 1;
const TARGET_FILE_SIZE_BASE: u64 = 64 * 1024 * 1024; // 64 MiB SST target
const MAX_BYTES_FOR_LEVEL_BASE: u64 = 512 * 1024 * 1024; // 512 MiB L1

#[derive(Debug, Error)]
pub enum StorageError {
    #[error("database error: {0}")]
    Database(String),
    #[error("serialization error: {0}")]
    Serialization(String),
}

/// Resident in-memory SafeBox cache plus the dirty account set.
///
/// The SafeBox is consensus-critical (its Merkle hash is committed into each
/// block header), but its *serialization* to RocksDB is only a cache: the same
/// state is reconstructible from the `accounts` column family. This cache keeps
/// the full SafeBox alive in memory and persists it only periodically, so a
/// block commit touches only the modified accounts instead of rewriting ~76 MB.
struct SafeboxCache {
    /// The resident SafeBox; `None` until first load.
    safebox: Option<SafeBox>,
    /// Resident index of in-use Ed25519 public keys → number of accounts
    /// currently using each key. Derived from the SafeBox accounts; used by
```

```rust
    /// `CreateAccount` to detect duplicate keys without an O(n) account scan.
    /// Ref-counted because multiple accounts can share a key (auto-created
    /// accounts) and `ChangeKey` moves a key between accounts.
    pubkey_index: HashMap<[u8; 32], u64>,
    /// AUGEID → sale listing, for accounts currently in `ForSale` state.
    sale_index: HashMap<u64, SaleListing>,
    /// AUGEID → pending gift, for accounts currently in `GiftPending` state.
    gift_index: HashMap<u64, GiftListing>,
    /// Account numbers modified since the last snapshot.
    dirty: std::collections::HashSet<u64>,
    /// Blocks committed since the last snapshot.
    blocks_since_snapshot: u64,
    /// Snapshot interval (blocks). 0 means snapshot on every commit.
    snapshot_interval: u64,
    /// Total full snapshots persisted.
    snapshot_total: u64,
    /// Bytes of the last persisted snapshot.
    last_snapshot_bytes: u64,
}

/// A marketplace listing for a single AUGEID.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct SaleListing {
    pub account_number: u64,
    pub seller_public_key: [u8; 32],
    pub price: u64,
    pub listed_at_block: u64,
}

/// A pending AUGEID gift awaiting the recipient's accept.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct GiftListing {
    pub account_number: u64,
    pub from_account_number: u64,
    pub recipient_public_key: [u8; 32],
    pub gifted_at_block: u64,
}

impl SafeboxCache {
    fn new() -> Self {
        SafeboxCache {
            safebox: None,
            pubkey_index: HashMap::new(),
            sale_index: HashMap::new(),
            gift_index: HashMap::new(),
            dirty: std::collections::HashSet::new(),
            blocks_since_snapshot: 0,
            snapshot_interval: 0,
```

```
            snapshot_total: 0,
            last_snapshot_bytes: 0,
        }
    }

    fn bump_pubkey(&mut self, pubkey: &[u8; 32], delta: i64) {
        let entry = self.pubkey_index.entry(*pubkey).or_insert(0);
        let next = (*entry as i64).saturating_add(delta).max(0) as u64;
        if next == 0 {
            self.pubkey_index.remove(pubkey);
        } else {
            *entry = next;
        }
    }

    /// Refresh the `sale_index`/`gift_index` entries for a single account based
    /// on its current state. Called on load and after every account mutation so
    /// the derived indices stay O(1) without ever scanning the whole SafeBox.
    fn sync_derived_indices(&mut self, account: &Account, block: u64) {
        let num = account.account_number;
        match account.account_info.state {
            augecoin_core::account::AccountState::ForSale => {
                self.sale_index.insert(
                    num,
                    SaleListing {
                        account_number: num,
                        seller_public_key: account.account_info.account_key.ed2551
                        price: account.account_info.price,
                        listed_at_block: block,
                    },
                );
            }
            _ => {
                self.sale_index.remove(&num);
            }
        }

        match account.account_info.state {
            augecoin_core::account::AccountState::GiftPending => {
// ... (1424 lines total)
```

Riscos

- WAL growth without proper configuration
- Snapshot staleness between checkpoint intervals
- No encryption at rest (relies on OS/disk encryption)

Referências Cruzadas

- **SafeBox** (Chapter 3): SafeBox state persisted via Storage
- **Consenso** (Chapter 4): Validator set stored in Storage

Chapter 6 — Network

> **Veja também:** Consenso, RPC

Executive Summary

AUGECOIN uses libp2p for P2P networking with Noise encryption, gossipsub for message propagation, and static bootnode discovery.

Responsibility

Peer-to-peer communication, transaction/block propagation, node synchronization.

Transport Layer

Noise Protocol encryption is mandatory — no unencrypted connections allowed.

```
libp2p Transport
  ├─ Noise (encryption)
  ├─ TCP (transport)
  └─ Yamux (multiplexing)
```

Gossipsub Topics

Topic	Content	Propagation
`tx`	Pending transactions	Flood to all peers
`block`	Finalized blocks	Flood to all peers
`consensus`	Consensus messages	Validator-only mesh

MAX_TRANSMIT_SIZE

Default libp2p limit: ~1MB. AUGECOIN blocks are expected to be well under this limit given the 60-second block time.

Peer Discovery

Static bootnode list (configurable). No Kademlia DHT — chosen for simplicity and auditability in a 4-validator network.

Sync Protocol

1. New node connects to bootnodes
2. Requests latest checkpoint from any peer
3. Restores state from checkpoint
4. Replays remaining blocks
5. Joins live consensus

Textual Flowchart

```
New Node
   │
   ├─ Connect to bootnodes (Noise handshake)
   │
   ├─ Request latest checkpoint
   │      └─ Restore SafeBox from checkpoint
   │
   ├─ Request blocks after checkpoint height
   │      └─ Execute each block (verify quorum)
   │
   ├─ Join live gossipsub mesh
   │      └─ Receive new blocks/tx
   │
   └─ Participate in consensus (if validator)
```

Key Source Files

Network Core

```rust
pub mod sync;
pub mod transport;

pub use libp2p::Multiaddr;
pub use libp2p::PeerId;

use libp2p::gossipsub;
use libp2p::identify;
use libp2p::kad;
use libp2p::multiaddr::Protocol;
use libp2p::noise;
use libp2p::ping;
use libp2p::swarm::NetworkBehaviour;
use libp2p::{identity, SwarmBuilder};
use libp2p_connection_limits as connection_limits;
use std::collections::HashSet;
use std::time::{Duration, Instant};
use thiserror::Error;
```

```rust
pub const OPS_TOPIC: &str = "augecoin/ops";
pub const BLOCK_TOPIC: &str = "augecoin/blocks";
pub const CONSENSUS_TOPIC: &str = "augecoin/consensus";
pub const PEERS_TOPIC: &str = "augecoin/peers";

/// Maximum size of a single gossipsub message (RPC payload), in bytes.
///
/// This bound must comfortably carry the largest legitimate consensus message
/// — a `BlockResponse` carrying a full block (Proposal/Prepare/Commit/Status
/// and gossiped operations are all strictly smaller). The value is read from
/// [`augecoin_core::limits::max_transmit_size`] so the g
```

Sync Manager

```rust
use augecoin_core::account::Account;
use augecoin_core::block::OperationBlock;
use augecoin_core::operation::{Operation, OperationPayload};
use augecoin_storage::checkpoint::AccountSnapshot;
use augecoin_storage::Storage;
use std::collections::{HashMap, VecDeque};
use thiserror::Error;

pub const SYNC_CHECKPOINT_TOPIC: &str = "augecoin/sync/checkpoint";
pub const SYNC_BLOCK_TOPIC: &str = "augecoin/sync/block";

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum SyncPhase {
    Idle,
    RequestingCheckpoint,
    RestoringCheckpoint,
    ReplayingBlocks,
    Complete,
}

#[derive(Debug, Clone, Error)]
pub enum SyncError {
    #[error("checkpoint deserialization failed: {0}")]
    InvalidCheckpoint(String),
    #[error("block deserialization failed: {0}")]
    InvalidBlock(String),
    #[error("storage error: {0}")]
    Storage(String),
    #[error("sync already in progress")]
    AlreadySyncing,
    #[error("no checkpoint received yet")]
    NoCheckpoint,
    #[error("block number mi
```

Riscos

- Static bootnodes = single point of initial connectivity
- No peer reputation scoring beyond rate limiting
- No encrypted storage of peer data

Referências Cruzadas

- **Consenso** (Chapter 4): Consensus messages travel over network
- **RPC** (Chapter 7): RPC is separate from P2P network

Chapter 7 — RPC

> **Veja também:** Wallet, CLI

Executive Summary

AUGECOIN exposes a JSON-RPC 2.0 API over TLS, with optional gRPC support. Authentication uses API keys for administrative endpoints.

RPC Methods

Method	Function	Auth
`getAccount`	Query account by number/address/name	Public
`getAccountByAddress`	Query by Bech32m address	Public
`getBlock`	Get block by hash	Public
`getBlockByHeight`	Get block by height	Public
`getBlockCount`	Current chain height	Public
`sendTransaction`	Submit signed transaction	Public
`getMempool`	List pending transactions	Public
`getValidatorSet`	Active validators	Public
`getNetworkStatus`	Node status, peers	Public
`getValidatorEarnings`	Validator rewards	Public
`listAccountsForSale`	Marketplace listings	Public
`resolveName`	Name → account lookup	Public
`buyAccount`	Purchase listed account	Public
`sellAccount`	List account for sale	Public
`giftAccount`	Initiate account gift	Public
`acceptGift`	Accept pending gift	Public
`validatorAdd`	Add validator	Admin
`validatorRemove`	Remove validator	Admin
`validatorActivate`	Activate validator	Admin
`validatorDeactivate`	Deactivate validator	Admin

Architecture Flow

AUGECOIN — RPC Map

> Fonte: `crates/augecoin-rpc/src/lib.rs`, `crates/augecoin-rpc/src/endpoints.rs`, `crates/augecoin-rpc/src/auth.rs`. Nós do grafo: `dispatch_method`, `AppState`, `check_auth`, `handle_*`.

Endpoints HTTP

Rota	Handler	Finalidade
`POST /`	`jsonrpc_handler`	JSON-RPC 2.0 genérico
`POST /createaccount`	`create_account_handler`	cria conta (sensível)
`POST /faucet`	`faucet_handler`	faucet (sensível)
`GET /health`	`health_handler`	liveness

Autenticação / rate limiting

- Header `x-api-key` → `auth::check_auth` → `AuthLevel::{Public, Admin}`.
- `ApiKeyStore` valida chaves conhecidas.
- `RateLimiter` global (100 req/60s) + `sensitive_rate_limiter` (5 req/60s) para `createaccount`/`faucet`.
- Métodos admin (`validator_*`) exigem `AuthLevel::Admin`.

Métodos JSON-RPC (`dispatch_method`)

Método	Handler (`endpoints.rs`)	Acesso
`getaccount`	`handle_get_account`	público
`createaccount`	`handle_create_account`	público (rate-limited)
`getblock`	`handle_get_block`	público
`getblockoperations`	`handle_get_block_operations`	público
`getoperations`	`handle_get_operations`	público
`getoperationbyhash`	`handle_get_operation_by_hash`	público
`getblockbyhash`	`handle_get_block_by_hash`	público
`getpendings`	`handle_get_pendings`	público
`sendoperation`	`handle_send_operation`	público
`findaccounts`	`handle_find_accounts`	público
`getaccountcount`	`handle_get_account_count`	público
`getblockcount`	`handle_get_block_count`	público
`getnodestatus`	`handle_get_node_status`	público
`getvalidatorset`	`handle_get_validator_set`	público
`validator_add`	`handle_validator_add`	admin
`validator_remove`	`handle_validator_remove`	admin
`validator_activate`	`handle_validator_activate`	admin

Método	Handler (endpoints.rs)	Acesso
validator_deactivate	handle_validator_deactivate	admin
listaccountsforsale	handle_list_accounts_for_sale	público
listvalidatorinventory	handle_list_validator_inventory	público
listpendinggifts	handle_list_pending_gifts	público
buyaccount	handle_buy_account	público (assinado)
sellaccount	handle_sell_account	público (assinado)
giftaccount	handle_gift_account	público (assinado)
acceptgift	handle_accept_gift	público (assinado)
cancelsale	handle_cancel_sale	público (assinado)

> **Nota sobre createaccount:** agora recebe account_number + public_key_hex e apenas *ativa* um AUGEID Reserved existente (assinado pelo admin/líder). Nunca cria números novos.

Marketplace e ciclo de vida do AUGEID

```
listaccountsforsale → Storage::list_for_sale() (sale_index residente)
listvalidatorinventory → Storage::safebox() filtrando por validator_public_key
buyaccount   → op BuyAccount  → Reserved/ForSale → Owned (atomic)
sellaccount  → op ListAccountForSale → (Reserved|Owned|Normal) → ForSale
giftaccount  → op GiftAccount → → GiftPending
acceptgift   → op AcceptGift  → GiftPending → Owned
cancelsale   → op DelistAccount → ForSale → Owned
```

Estado compartilhado (AppState)

```
AppState {
  storage: Arc<Storage>              // RocksDB
  mempool: Arc<Mutex<Mempool>>       // pool de operações pendentes
  node_status: Arc<NodeStatus>       // métricas/status
  faucet_keypair, faucet_account, faucet_amount
  faucet_claims: Mutex<HashMap>      // claims por endereço (24h)
  api_keys: ApiKeyStore
  rate_limiter, sensitive_rate_limiter
  admin_keypair
  op_broadcast: sender               // gossip p/ operações admitidas
}
```

Fluxo de sendoperation (caminho até o consenso)

```
POST / (sendoperation)
  → dispatch_method → handle_send_operation
  → valida assinatura + nonce + saldo
  → mempool.validate_and_admit
  → op_broadcast (gossip para demais validadores)
```

→ (no próximo bloco) execute_block consome do mempool

Wallet Web — quais RPCs o frontend consome

O frontend (`apps/wallet-web/src/services/rpc.ts`) é a **única** porta de acesso à blockchain e consome **apenas** os métodos oficiais abaixo — não há RPC inventada na UI nem regra de consenso duplicada:

- Consulta: `getaccount`, `findaccounts`, `listaccountsforsale`, `listvalidatorinventory`, `listpendinggifts`, `getnodestatus`.
- Ciclo de vida (assinado): `buyaccount`, `sellaccount`, `giftaccount`, `acceptgift`, `cancelsale`.
- Operações brutas: `sendoperation`.

  O `server/` da Wallet Web **não** é uma RPC da blockchain: ele é um serviço de identidade/registro (Conta da Plataforma + `linked_wallets`), exposto em `/api/*` (HTTP + cookies), que nunca detém saldo, estado ou chaves privadas. Ele apenas responde "quais AUGEIDs pertencem a este usuário?" — o saldo/nome/ estado de cada AUGEID é sempre lido do SafeBox via `getaccount`.

Endpoint Implementation

```rust
use augecoin_consensus::validator::{ValidatorSet, ValidatorStatus};
use augecoin_core::account::Account;
use augecoin_core::account::AccountState;
use augecoin_core::block::OperationBlockHeader;
use augecoin_core::mempool::Mempool;
use augecoin_core::operation::{Operation, OperationPayload, ReceiverInfo, SenderIn
use augecoin_storage::Storage;
use serde::{Deserialize, Serialize};
use std::collections::HashMap;
use std::sync::atomic::{AtomicU32, AtomicU64};
use std::sync::Arc;
use std::time::{SystemTime, UNIX_EPOCH};

use crate::auth::{ApiKeyStore, RateLimiter};

/// Shared runtime status snapshot, updated by the node main loop and
/// read by the RPC layer. All fields use interior mutability so that
/// the node can update the snapshot without re-locking the AppState.
#[derive(Debug)]
pub struct NodeStatus {
    pub block_height: AtomicU64,
    pub latest_block_hash: std::sync::Mutex<[u8; 64]>,
    pub peers_connected: AtomicU32,
    /// Peers in the gossipsub mesh of the consensus topics (the peers that
    /// actually exchange consensus messages).
```

```
    pub peers_gossipsub_consensus: AtomicU32,
    /// Peers known via Kademlia DHT discovery (not necessarily connected).
    pub peers_kademlia_total: AtomicU32,
    pub syncing: AtomicU64,
    pub sync_target_height: AtomicU64,
    pub mempool_size: AtomicU64,
    pub current_round: AtomicU64,
    pub current_view: AtomicU64,
    pub validator_id: AtomicU64,
    pub chain_id: AtomicU64,
    pub uptime_seconds: AtomicU64,
    pub last_consensus_error: std::sync::Mutex<Option<String>>,
}

impl Default for NodeStatus {
    fn default() -> Self {
        NodeStatus {
            block_height: AtomicU64::new(0),
            latest_block_hash: std::sync::Mutex::new([0u8; 64]),
            peers_connected: AtomicU32::new(0),
            peers_gossipsub_consensus: AtomicU32::new(0),
            peers_kademlia_total: AtomicU32::new(0),
            syncing: AtomicU64::new(0),
            sync_target_height: AtomicU64::new(0
```

Authentication

```
use serde::Deserialize;
use std::collections::HashMap;
use std::sync::Mutex;
use std::time::{Duration, Instant};

#[derive(Debug, Clone)]
pub struct ApiKeyStore {
    keys: HashMap<String, KeyInfo>,
}

#[derive(Debug, Clone)]
struct KeyInfo {
    is_admin: bool,
}

impl KeyInfo {
    fn new(is_admin: bool) -> Self {
        KeyInfo { is_admin }
    }
}
```

```rust
impl ApiKeyStore {
    pub fn new(admin_keys: Vec<String>) -> Self {
        let mut keys = HashMap::new();
        for key in admin_keys {
            keys.insert(key, KeyInfo::new(true));
        }
        ApiKeyStore { keys }
    }

    pub fn empty() -> Self {
        ApiKeyStore {
            keys: HashMap::new(),
        }
    }

    pub fn is_valid(&self, key: &str) -> bool {
        self.keys.contains_key(key)
    }

    pub fn is_admin(&self, key: &str) -> bool {
        self.keys.get(key).is_some_and(|info| info.is_admin)
    }
}

#[derive(Debug)]
struct ClientBucket {
    tokens: u32,
    last_refill: Instant,
}

impl ClientB
```

RPC Server Core

```rust
pub mod auth;
pub mod config;
pub mod endpoints;

use axum::{
    routing::{get, post},
    Json, Router,
};
use axum_server::tls_rustls::RustlsConfig;
use serde::{Deserialize, Serialize};
use std::net::SocketAddr;
use std::sync::Arc;
```

```rust
use thiserror::Error;
use tokio::net::TcpListener;

#[derive(Debug, Error)]
pub enum RpcError {
    #[error("TLS configuration error: {0}")]
    TlsConfig(String),
    #[error("server error: {0}")]
    Server(String),
    #[error("io error: {0}")]
    Io(#[from] std::io::Error),
}

#[derive(Debug, Clone)]
pub struct RpcSettings {
    pub jsonrpc_addr: SocketAddr,
}

impl Default for RpcSettings {
    fn default() -> Self {
        RpcSettings {
            jsonrpc_addr: "127.0.0.1:0".parse().unwrap(),
        }
    }
}

// ── JSON-RPC 2.0 types ──────────────────────────────────────────

#[derive(Debug, Deserialize)]
pub struct JsonRpcRequest {
    pub jsonrpc: String,
    pub method: String,
    #[serde(default)]
    pub params: serde_json::Value,
    pub id: serde_json::Value,
}

#[derive(Debug, Serialize)]
pub struct JsonRpcResponse {
    pub jsonrpc: String,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub result: Option<serde_json::Value>,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub error: Option<JsonRpcError>,
    pub id: serde_json::Value,
}

#[derive(Debug, Serialize, Clone)]
```

```rust
pub struct JsonRpcError {
    pub code: i32,
    pub message: String,
}

type AppState = endpoints::AppState;

/// Methods that mint tokens or create state and are prime spam targets on a
/// public testnet. These are additionally throttled by the stricter
/// `sensitive_rate_limiter` on top of the global limiter.
const SENSITIVE_METHODS: &[&str] = &["createaccount", "faucet"];

fn client_identifier(
    api_key: Option<&str>,
    remote: Option<SocketAddr>,
    forwarded_for: Option<&str>,
) -> String {
    if let Some(key) = api_key {
        return format!("key:{key}");
    }
    // Behind a reverse proxy the TCP peer
```

Riscos

- TLS certificate management
- API key rotation
- Rate limiting effectiveness under DDoS

Referências Cruzadas

- **Wallet** (Chapter 8): Wallet uses RPC for all blockchain interactions
- **CLI** (Chapter 10): CLI wraps RPC calls

Chapter 8 — Wallet

> **Veja também:** RPC, AUGEID

Executive Summary

AUGECOIN provides three wallet implementations: Web (React), Desktop (Tauri), and Mobile (React Native). All use the same cryptographic core.

Platform Account vs Blockchain Wallet

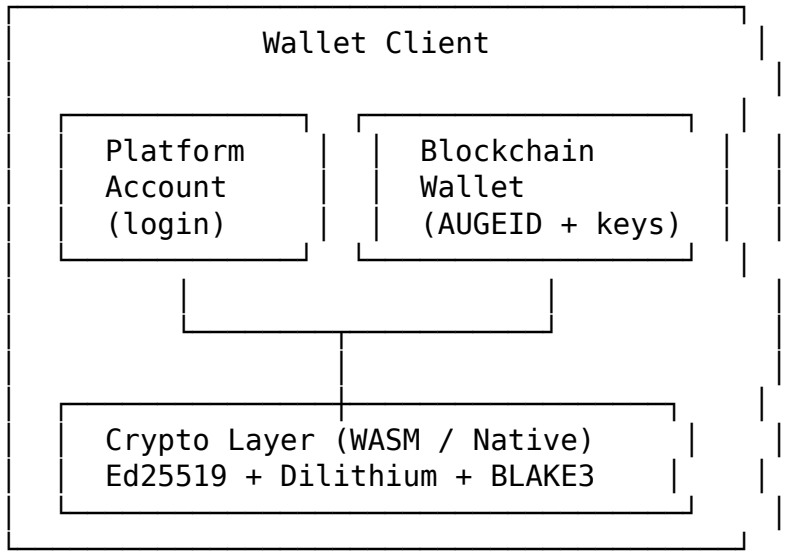
Platform Account

- Login, profile, session management
- Server-side (Express.js)
- JWT-based authentication
- Stores user preferences

Blockchain Wallet

- AUGEID identity
- Balance, send/receive
- All crypto operations client-side
- Never sends private keys to server

Wallet Architecture



Marketplace

Accounts can be listed for sale, gifted, or transferred: - `sellAccount`: List at a price - `buyAccount`: Purchase listed account - `giftAccount`: Send as gift (pending acceptance) - `acceptGift`: Accept pending gift

Inventory

Each wallet tracks: - Owned accounts - Pending gifts - Listed accounts (for sale) - Gift history

Key Files

Wallet	Framework	Crypto
wallet-web	React + Vite	WASM (@augecoin/wasm-crypto)
wallet-desktop	Tauri (Rust)	Native (augecoin-crypto)
wallet-mobile	React Native	Native module

Riscos

- Seed backup is user responsibility
- No account recovery mechanism
- Keystore encryption depends on user password strength

Referências Cruzadas

- **RPC** (Chapter 7): All wallet operations go through RPC
- **AUGEID** (Chapter 9): Account lifecycle managed by wallets

---

Chapter 9 — AUGEID

> **Veja também:** SafeBox, Wallet, Consenso

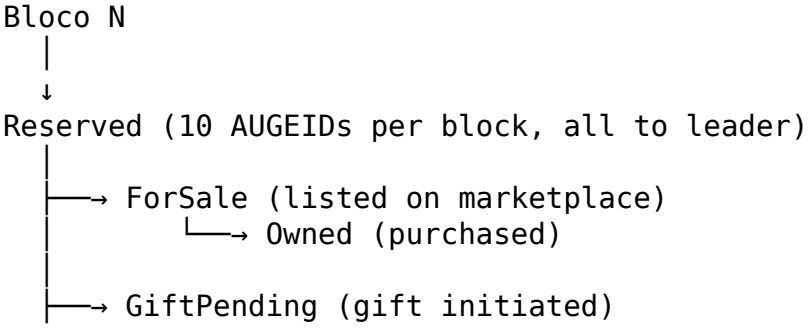
Executive Summary

AUGEID is AUGECOIN's native account identity system — numbered accounts that serve as both blockchain addresses and tradeable assets.

Responsibility

Provide deterministic, sequential account numbering with full lifecycle management (creation, sale, gifting, naming).

Account Lifecycle



```
│       └──→ Owned (gift accepted)
│
└──→ Owned (direct activation)
        └──→ Normal (operational account)
```

Account States

State	Description
`Reserved`	Emitted to block leader; no key; cannot send/receive
`Owned`	Has owner (account_key, n_operation=0, account_seal)
`ForSale`	Listed on marketplace (sale_index)
`GiftPending`	Gift pending acceptance (gift_index)
`Normal`	Fully operational account

Numbering Formula

AUGEID = block × 10 + offset (0..9)

Each block emits exactly 10 AUGEIDs, all `Reserved` and belonging to the leader.

Why AUGEID is an Asset

1. **Scarcity**: Only 10 per block, fixed emission schedule
2. **Tradeability**: Can be bought, sold, gifted
3. **Identity**: Named accounts (resolve_name)
4. **Speculation**: Early/lower numbers may have premium value
5. **No expiration**: Accounts persist forever

Key Invariants

- `SafeBox.accounts` ≡ CF `accounts` after commit
- `name_index` ≡ derived from `account.name`
- Emission is fixed: exactly 10 `Reserved` per block
- `CreateAccount` never creates new numbers — only activates existing `Reserved`

Riscos

- Leader controls all 10 AUGEIDs per block (centralization of issuance)
- No mechanism to recover lost keys
- Name squatting possible

Referências Cruzadas

- **SafeBox** (Chapter 3): AUGEIDs stored in SafeBox
- **Wallet** (Chapter 8): Wallet manages AUGEID lifecycle
- **Consenso** (Chapter 4): Block leader receives AUGEIDs

Chapter 10 — Benchmark

Performance Metrics

Metric	Target	Status
350,000 transactions	Stress test	✓
117 TPS	Throughput	✓
0 rejections	Integrity	✓
4 validators	Quorum	✓
60s block time	Block interval	✓
120s timeout	View-change	✓

Benchmark Files

- `crates/augecoin-bench/` — Rust benchmarks
- `crates/augecoin-node/tests/load_test.rs` — Load testing
- `crates/augecoin-node/tests/partition_test.rs` — Network partition

Test Categories

Unit Tests

- Per-crate test suites
- Serialization round-trips
- Cryptographic verification

Integration Tests

- E2E transaction lifecycle
- 4-node liveness
- Determinism verification
- SDK parity (Rust ↔ TypeScript)

Adversarial Tests

- Equivocation detection
- Duplicate vote rejection
- Invalid proposal handling
- Quorum edge cases

Property Tests

- Emission total supply verification (proptest)
- Block reward sum = TOTAL_SUPPLY_AUGESAT

Riscos

- Benchmarks run on single machine, not distributed
- Real network latency not simulated
- Storage benchmarks may vary by hardware

Referências Cruzadas

- **Segurança** (Chapter 11): Security tests complement benchmarks
- **Testes** (Chapter 12): Complete test inventory

Chapter 11 — Security

> **Veja também:** Consenso, Rede, Storage

Executive Summary

AUGECOIN implements defense-in-depth with hybrid cryptography, atomic writes, replay protection and on-chain equivocation proofs.

Cryptographic Security

Primitive	Algorithm	Purpose
Signature	Ed25519 + Dilithium (dual)	Transaction/block signing
Hashing	BLAKE3-512 (XOF)	State root, block hash
Key Derivation	HKDF-SHA3-512	HD wallet derivation
Address	Bech32m	Human-readable addresses

Replay Protection

- op_sequence per account: monotonically increasing
- Same operation cannot be submitted twice
- Mempool rejects stale sequences

Atomicity

All block execution uses RocksDB WriteBatch: - Either ALL changes commit, or NONE - No partial state visible - Crash recovery from last committed state

AUGECOIN — Security Surface

Mapeamento da superfície de ataque a partir do grafo (`graph.json`) e leitura de `rpc/lib.rs`, `rpc/auth.rs`, `execution.rs`, `consensus.rs`, `crypto/*`.

Superfícies de ataque

1. RPC HTTP (porta configurável, exposta na rede)

- `POST /`, `/createaccount`, `/faucet`, `GET /health`.
- Vetores: rate-limit (mitigado por `RateLimiter` global + sensível), auth (métodos admin exigem `x-api-key` válido), `createaccount`/`faucet` são sensíveis e têm limitador dedicado de 5 req/60s.

2. libp2p / rede (gossipsub + Kademlia)

- Vetores: nós maliciosos, mensagens oversized (mitigado por `max_transmit_size=1MiB`), spoofing de endereço (mitigado por `AUGECOIN_EXTERNAL_ADDRESS` e rejeição de `0.0.0.0`/`::`).
- `deserialize_consensus_msg`, `read_u32_be`/`read_u64_be` — limites de comprimento são críticos para evitar OOM/overflow.

3. Validação de assinaturas

- `verify_operation_signatures` (Ed25519) sobre `to_bytes_stripped()`.
- `verify_block_quorum`: assinatura do líder + quórum 2/3 + `operations_hash`.
- `Ed25519KeyPair`/`HybridSignature` — nunca usar chave com verificação frouxa.

4. Serialização crítica

- `Account::to_bytes/from_bytes`, `Operation::to_bytes_stripped`, `SafeBox::to_bytes/fr`
  `OperationBlock::from_bytes`, `ValidatorSet::to_bytes/from_bytes`.
- Vetor: dados malformados → `from_bytes` deve falhar fechado (retornam `Option`/`Result`, nunca panic em input externo).

5. Faucet

- `faucet_claims` persistido em RocksDB (`faucet_claims` CF); 1 claim por AUGEID por 24h. `faucet_account` controlado por `faucet_keypair`.
- Destino por `account_number` (AUGEID); `Reserved`/`GiftPending` não recebem.

6. Marketplace e doação (novos vetores)

- `buyaccount`/`sellaccount`/`giftaccount`/`acceptgift`/`cancelsale` são assinados (Ed25519) e validados em `verify_operation_signatures`.

- `CreateAccount` exige assinatura do admin/líder e só ativa AUGEID `Reserved`.
- Proteção anti-replay via `n_operation`.
- Regras de consenso: só o proprietário vende; só o líder recebe emissão; `Reserved`/`GiftPending` não movimentam saldo; dupla venda/doação são rejeitadas pelo estado da conta.

7. Chaves e secrets

- `AUGECOIN_VALIDATOR_KEY_HEX` / `_FILE`, `AUGECOIN_FAUCET_KEY_HEX`.
- Nunca logar chaves; `genesis.rs` valida `public_key != [0u8;32]`.

Caminhos RPC privilegiados (admin)

`validator_add`, `validator_remove`, `validator_activate`, `validator_deactivate` → exigem `AuthLevel::Admin` via `check_auth`. Qualquer bypass do `ApiKeyStore` é uma escalada de privilégio crítica (controle do conjunto de validadores). `createaccount` é assinado pelo admin (não cria números, só ativa `Reserved`).

Código sensível

- `crypto/signature.rs` — Ed25519 + assinatura híbrida.
- `crypto/hash.rs` — `blake3_512`.
- `crypto/address.rs` (bech32) — `derive_address`/`validate_address`.
- `crypto/address.rs` (Base58Check curto) — `derive_short_address` é somente uma representação externa; o endereço canônico Bech32m continua sendo a identidade usada pelo protocolo.
- `core/safe_box.rs` — Merkle proof + hash de consenso.
- `node/execution.rs` — execução determinística + emissão (hard cap).

Wallet Web — superfície do backend de plataforma (`apps/wallet-web/server`)

Serviço separado da blockchain (HTTP `/api/*`). Nunca detém saldo, estado do SafeBox ou chave privada — apenas identidade + registro de vínculos.

Controle	Implementação
Senha	`crypto.scrypt` (N=16384, r=8, p=1), salt por usuário, `timingSafeEqual`
Sessão	JWT access (15 min, HttpOnly/SameSite=Lax) + refresh token opaco rotativo, armazenado como `sha256`
Logout global	revoga todos os refresh tokens do usuário
CSRF	double-submit (`X-CSRF-Token` vs cookie não-HttpOnly) + verificação de Origin
Rate limit login	em memória: 5/60s por IP e por e-mail

Controle	Implementação
Chave privada	**nunca** enviada ao backend; o mnemônico é gerado e criptografado (IndexedDB keystore, Argon/PBKDF2 + AES-GCM) apenas no cliente
Registro	cria somente `platform_users` + `user_preferences` — nunca um AUGEID

Tabelas: `platform_users`, `user_preferences`, `linked_wallets`, `refresh_tokens`. `linked_wallets` é a camada de ponte (Wallet Registry); a autoridade sobre propriedade/saldo/transferência continua exclusiva do SafeBox.

Pré-requisitos de produção: definir `AUGECOIN_WALLET_JWT_SECRET` e `AUGECOIN_WALLET_ORIGINS`; ativar `AUGECOIN_WALLET_SECURE_COOKIES=1` sob TLS. Suporte futuro planejado: WebAuthn e 2FA (sem alterar o consenso).

Recomendações

1. Fuzz continuado em `from_bytes` (existe `crates/augecoin-core/fuzz`).
2. Manter `rate_limiter`/`sensitive_rate_limiter` calibrados sob carga real.
3. Revisar `deserialize_consensus_msg` para limites de tamanho de cada campo.
4. Monitorar `augecoin_equivocation_events_total` (equivocação) via Prometheus.

Threat Model

AUGECOIN — Threat Model

This document catalogs known threats to the AUGECOIN network and the defenses in place (or planned) for each.

Threat: Byzantine Validator (Equivocation)

- **Attack:** A validator signs and broadcasts two conflicting blocks at the same height, attempting to fork the chain.
- **Impact:** Network fork; clients may see inconsistent state; double-spend window opens.
- **Existing Defense:** Built-in equivocation detection — duplicate votes at the same height generate an equivocation proof that is gossiped and logged. Alerts fire via Prometheus and CLI.
- **Missing Defense:** Automatic slashing. The protocol detects equivocation but relies on the admin to manually deactivate the offender.
- **Mitigation:** Monitor equivocation alerts; run `augecoin-cli security equivocations` on every alert; have a pre-signed deactivation transaction ready.

Threat: Malicious Peer (Spam / Gossip Abuse)

- **Attack:** A peer floods the gossip mesh with invalid messages (empty blocks, malformed transactions, duplicate votes) to waste bandwidth and CPU.
- **Impact:** Degraded performance for honest validators; increased mempool pressure; potential for eclipse attacks on specific nodes.
- **Existing Defense:** libp2p Noise authentication ensures only peers with valid Ed25519 identities can join the mesh. Invalid messages are dropped before gossip relay.
- **Missing Defense:** No peer reputation scoring. A peer that sends 10 000 invalid messages is treated the same as one that sends none.
- **Mitigation:** Operators should configure firewall rules to rate-limit incoming P2P connections per IP. Consider a future peer-scoring subsystem that banishes repeat offenders.

Threat: Stolen Validator Key

- **Attack:** An attacker gains access to a validator's Ed25519 private key through filesystem compromise, backup exfiltration, or supply-chain attack.
- **Impact:** Attacker can sign conflicting votes (equivocate), stall cons

Security Audit

AUGECOIN — Security Policy

Reporting a Vulnerability

If you discover a security vulnerability in AUGECOIN, please do **not** open a public issue. Instead, send an encrypted report to:

- **Email:** [security@augecoin.org]
- **PGP Key:** [TODO: publish PGP public key fingerprint]

We aim to acknowledge reports within 48 hours and provide an initial assessment within 7 days. Please include as much detail as possible: affected component, steps to reproduce, and potential impact.

Supported Versions

Version	Status
`v1.x` (latest)	:white_check_mark: Supported
`v0.x` (pre-mainnet)	:x: End of life

Only the latest stable release line receives security patches. Pre-mainnet releases are unsupported and may contain known vulnerabilities.

Security Model

Consensus Security

AUGECOIN uses a Proof-of-Authority consensus based on Byzantine Fault Tolerance (BFT). The validator set is fixed at **4 validators** and a quorum of **2/3 + 1** (i.e., 3 out of 4) is required to finalize a block.

- A single malicious validator cannot halt or corrupt the chain.
- Two colluding validators can stall finality but cannot forge committed blocks, because 3 votes are needed for a commit.
- Validators that equivocate (sign conflicting blocks at the same height) are detected automatically and an alert is raised. The admin can deactivate and replace a misbehaving validator on-chain.

Key Management

AUGECOIN separates cryptographic keys by role:

Key	Purpose
Validator key	Sign consensus votes (pre-prepare, prepare, commit)
Admin key	Validator-set management (add/remove validators), protocol upgrades
User key	Sign transactions (transfer, mint, burn)

- Validator ke

Known Limitations

- No atomic swaps
- No blockchain deletion
- No coin-recovery from lost keys
- Admin key is single-key (not multisig)

Riscos

- Centralized validator management
- No formal verification
- Dilithium key sizes increase transaction size

Referências Cruzadas

- **Consenso** (Chapter 4): Quorum prevents Byzantine behavior
- **Rede** (Chapter 6): Noise encryption prevents network attacks

- **Storage** (WriteBatch): Atomicity prevents partial writes

---

Chapter 12 — Tests

Test Inventory

Cargo Test Suites

Crate	Tests	Type
augecoin-crypto	Signature, hash, hdkeys, address	Unit
augecoin-core	Account, block, transaction, emission	Unit + Property
augecoin-storage	RocksDB round-trip, checkpoint, state_root	Integration
augecoin-consensus	Quorum, round, validator, equivocation	Unit + Adversarial
augecoin-network	Transport, gossip, sync, rate_limit	Integration
augecoin-node	Mempool, execution, genesis, e2e	Integration
augecoin-rpc	Endpoints, auth, TLS	Integration
augecoin-cli	Commands, admin auth	Integration

Integration Tests (crate-level)

- `crates/augecoin-node/tests/e2e_transaction.rs` — Full transaction lifecycle
- `crates/augecoin-node/tests/end_to_end_block_execution.rs` — Block execution
- `crates/augecoin-node/tests/determinism_test.rs` — Determinism verification
- `crates/augecoin-consensus/tests/four_node_liveness.rs` — 4-node consensus
- `crates/augecoin-consensus/tests/adversarial_tests.rs` — Attack scenarios
- `crates/augecoin-node/tests/sdk_parity.rs` — Cross-language parity
- `crates/augecoin-node/tests/load_test.rs` — Performance testing
- `crates/augecoin-node/tests/partition_test.rs` — Network partition

Fuzz Targets

- `crates/augecoin-core/fuzz/fuzz_targets/tx_deserialize.rs`
- `crates/augecoin-core/fuzz/fuzz_targets/block_deserialize.rs`

Clippy

```
cargo clippy --workspace --all-targets
```

Graphify Analysis

Graphify provides semantic analysis of the codebase: - 3167 nodes, 6966 edges, 167 communities - 99% extraction confidence - Automatic architecture documentation

Test Commands

```
# All tests
cargo test --workspace

# Specific crate
cargo test -p augecoin-crypto
cargo test -p augecoin-core
cargo test -p augecoin-consensus

# Linting
cargo clippy --workspace --all-targets

# Formatting
cargo fmt --all -- --check

# Benchmarks
cargo bench --workspace

# Fuzzing
cargo fuzz run tx_deserialize -- -max_total_time=300
```

Riscos

- No CI/CD pipeline in repository
- Fuzzing coverage limited
- No mutation testing

Referências Cruzadas

- **Benchmark** (Chapter 10): Performance metrics
- **Segurança** (Chapter 11): Security test categories

Chapter 13 — APIs

JSON-RPC 2.0 Interface

Base URL: `https://<node>:9443` (TLS required)

Request Format

```
{{
  "jsonrpc": "2.0",
  "method": "getAccount",
  "params": [42],
  "id": 1
}}
```

Response Format

```
{{
  "jsonrpc": "2.0",
  "result": {{
    "account_number": 42,
    "balance": 1000000000,
    "name": "MyAccount",
    "state": "Normal"
  }},
  "id": 1
}}
```

Method Reference

getAccount

Parameters: - `account_number` (u64): Account number, OR - `address` (string): Bech32m address, OR - `name` (string): Account name

Returns: AccountInfo object

Example:

```
curl -k https://localhost:9443 \
  -d '{{"jsonrpc":"2.0","method":"getAccount","params":[42],"id":1}}'
```

sendTransaction

Parameters: - `transaction` (hex): Signed transaction bytes

Returns: SendOperationResult

Example:

```
curl -k https://localhost:9443 \
  -d '{{"jsonrpc":"2.0","method":"sendTransaction","params":["0x..."],"id":1}}'
```

buyAccount

Parameters: - `account_number` (u64): Account to buy - `price` (u64): Price in augesat

Returns: Operation result

sellAccount

Parameters: - `account_number` (u64): Account to list - `price` (u64): Price in augesat

Returns: Operation result

giftAccount

Parameters: - `account_number` (u64): Account to gift - `recipient` (string): Recipient address

Returns: Operation result (GiftPending)

acceptGift

Parameters: - `account_number` (u64): Gifted account to accept

Returns: Operation result (Owned → Normal)

listAccountsForSale

Parameters: None

Returns: Array of marketplace listings

resolveName

Parameters: - `name` (string): Account name to resolve

Returns: Account number

gRPC Interface

Proto definition available in `crates/augecoin-rpc/proto/`.

Service: `augecoin.v1.NodeService`

Endereço Curto Externo

O protocolo mantém `auge1...` como endereço canônico. Para pagamento e exibição, o SDK oferece `getShortAddress()` e `validateShortAddress()`. O formato usa Base58Check com payload de 24 bytes derivado de BLAKE3 e checksum de 4 bytes, resultando em aproximadamente 39 caracteres.

```
const canonical = getAddress(publicKey);       // identidade interna
const short = getShortAddress(publicKey);      // pagamento/exibição
validateShortAddress(short);                   // valida checksum
```

O endereço curto não substitui nem altera endereços canônicos existentes.

Riscos

- No API versioning
- No request batching
- No WebSocket subscription for real-time updates

Referências Cruzadas

- **RPC** (Chapter 7): Server implementation details
- **Wallet** (Chapter 8): Client-side API usage

Chapter 14 — Annexes

Project Tree

```
opt/augecoin/
  Cargo.toml (workspace)
  crates/augecoin-bench/
    client.rs
    exporters.rs
    lib.rs
    main.rs
    metrics.rs
    ... (10 files)
  crates/augecoin-cli/
    commands.rs
    main.rs
    output.rs
    rpc.rs
  crates/augecoin-consensus/
    equivocation.rs
    lib.rs
    quorum.rs
    round.rs
```

```
  validator.rs
crates/augecoin-core/
  account.rs
  block.rs
  constants.rs
  emission.rs
  lib.rs
  ... (12 files)
crates/augecoin-crypto/
  address.rs
  hash.rs
  hdkeys.rs
  lib.rs
  signature.rs
  ... (6 files)
crates/augecoin-network/
  lib.rs
  sync.rs
  transport.rs
crates/augecoin-node/
  alerts.rs
  keygen.rs
  consensus.rs
  execution.rs
  genesis.rs
  ... (8 files)
crates/augecoin-rpc/
  auth.rs
  config.rs
  endpoints.rs
  lib.rs
crates/augecoin-storage/
  checkpoint.rs
  lib.rs
  state_root.rs
docs/
  SPEC.md
  DECISIONS.md
  architecture/
graphify-out/
  graph.json
  GRAPH_REPORT.md
apps/
  wallet-web/
  wallet-desktop/
  wallet-mobile/
packages/
  sdk-ts/
```

```
    wasm-crypto/
  infra/
    testnet-4node.yml
    observability/
  tools/
    technical-book/
```

Build Metadata

Field	Value
Generated	2026-08-19 13:26 UTC
Git Commit	unknown
Git Version	unknown
Rust Version	rustc 1.97.1 (8bab26f4f 2026-07-14)
Graphify	data present

Cargo Workspace

```toml
[workspace]
resolver = "2"
members = [
    "crates/augecoin-crypto",
    "crates/augecoin-core",
    "crates/augecoin-storage",
    "crates/augecoin-consensus",
    "crates/augecoin-network",
    "crates/augecoin-rpc",
    "crates/augecoin-cli",
    "crates/augecoin-node",
    "crates/augecoin-bench",
    "apps/wallet-desktop/src-tauri",
]
```

Protocol Constants

Constant	Value
TOTAL_SUPPLY_AUGE	750,000,000
TOTAL_SUPPLY_AUGESAT	75,000,000,000,000,000
DECIMALS	8
BLOCK_TIME_SECONDS	60
EMISSION_YEARS	50
TOTAL_EMISSION_BLOCKS	26,298,000
BASE_REWARD_AUGESAT	2,852,383
QUORUM	2/3 + 1

Emission Schedule

```
block_reward(h) for h in 1..=26,298,000:
  base = 2,852,383 augesat
  if h <= 10,266 (REMAINDER):
    reward = base + 1
  else:
    reward = base
  (after TOTAL_EMISSION_BLOCKS: reward = 0)
```

Sum verification: `sum(block_reward(h)) == TOTAL_SUPPLY_AUGESAT` exactly.